UNIVERSIDAD NACIONAL EXPERIMENTAL DE TÁCHIRA
VICERRECTORADO ACADÉMICO
DECANATO DE DOCENCIA
DEPARTAMENTO DE INGENIERÍA INFORMÁTICA
REDES NEURONALES

Evaluación #6 – Implementar ANN con Matlab

Autor:

Juan Diego Paredes Gámez V-31.357.791

Sección: 01

San Cristóbal, abril de 2026

1. Objetivos

1.1 Objetivo General

Implementar y comparar tres arquitecturas de redes neuronales artificiales distintas (*Reconocimiento de Patrones, Regresión y Mapa Autoorganizado*) sobre tres datasets canónicos del Neural Network Toolbox de MATLAB, analizando el efecto de diferentes configuraciones de entrenamiento sobre el rendimiento de cada modelo.

1.2 Objetivos Específicos

- Aplicar una *red de reconocimiento de patrones* (**patternnet**) al dataset cancer_dataset y comparar el desempeño de los algoritmos trainlm y trainscg.
- Emplear una *red de ajuste/regresión* (**fitnet**) sobre bodyfat_dataset evaluando el impacto de la regularización bayesiana (trainbr) frente a Levenberg-Marquardt (trainlm).
- Entrenar un *Mapa Autoorganizado* (**selforgmap**) sobre iris_dataset con dos topologías (2×3 y 4×3) para comparar la resolución de clustering resultante.
- **Documentar y discutir** el efecto de los hiperparámetros sobre el sobreajuste, la velocidad de convergencia y la calidad de los resultados.

2. Instrucciones de Ejecución

Para replicar los resultados de esta evaluación, se requiere MATLAB **(R2018b o superior)** con el Neural Network Toolbox instalado.

1. Verifique la disponibilidad de los datasets ejecutando en la ventana de comandos: [X,T] = cancer_dataset(); (repita para bodyfat_dataset e iris_dataset).

2. Agregue los scripts proporcionados al path de MATLAB o asegúrese de que el Current Folder sea el directorio que contiene los archivos .m.

3. Ejecute cada script de forma independiente desde la ventana de comandos: caso1_cancer_patternnet, caso2_bodyfat_fitnet, caso3_iris_som.

4. Cada script imprimirá en consola un bloque de resumen comparativo y generará automáticamente las ventanas (Figures) con las gráficas de rendimiento correspondientes.

Nota: El parámetro net.trainParam.showWindow = false suprime las ventanas GUI de entrenamiento. Cambiarlo a true permite visualizar la curva de error en tiempo real.

3. Marco Teórico

3.1 Red de Reconocimiento de Patrones (patternnet)

Una **patternnet** es una red neuronal multicapa feedforward *(arquitectura clásica unidireccional, sin ciclos ni retroalimentación de la información)* entrenada para clasificación. La capa de salida utiliza la función de activación softmax, produciendo una distribución de probabilidad sobre las clases, y el error se mide con la entropía cruzada. Esta arquitectura es idónea para el cancer_dataset por ser un problema de clasificación binaria (tumor benigno vs. maligno). La función de error por entropía cruzada penaliza fuertemente las predicciones incorrectas con alta confianza, haciéndola superior al MSE para este fin ***(Hagan et al., 2014)***.

3.2 Red de Ajuste (fitnet)

Una **fitnet** es una red de regresión que aproxima una función continua y = f(x). Utiliza funciones de activación tansig en capas ocultas y purelin en la salida, optimizando el Error Cuadrático Medio (MSE). Se eligió para el bodyfat_dataset debido a que el objetivo es predecir una variable continua —el porcentaje de grasa corporal— a partir de trece variables antropométricas, donde las métricas naturales de evaluación son el MSE y el coeficiente de determinación R^2 ***(MathWorks, 2024)***.

3.3 Mapa Autoorganizado (selforgmap)

Un **SOM** (Self-Organizing Map) es una red neuronal no supervisada que proyecta datos de alta dimensión sobre una cuadrícula 2D preservando la topología del espacio original mediante aprendizaje competitivo de Kohonen (2001). Se seleccionó para el iris_dataset porque el objetivo es descubrir la estructura natural de agrupamiento de las tres especies sin utilizar las etiquetas de clase durante la fase de entrenamiento, evaluando la calidad del resultado a posteriori mediante la métrica de Pureza.

4. Metodología y Parámetros de Entrenamiento

4.1 Caso 1: cancer_dataset

Se compararon dos configuraciones sobre la misma arquitectura de clasificación. La selección de parámetros se fundamentó en el contraste entre un optimizador de segundo orden (trainlm) y uno de primer orden más eficiente en memoria (trainscg), combinado con la variación del número de neuronas en la capa oculta.

Parámetro	Configuración A	Configuración B
Función de entrenamiento	trainlm **(Levenberg-Marquardt)**	trainscg **(Gradiente Conjugado Escalado)**
Neuronas en capa oculta	10	20
División Train/Val/Test	70% / 15% / 15%	70% / 15% / 15%
Épocas máximas	500	500
Objetivo de error	1×10^{-5}	1×10^{-5}
Early-stopping (max_fail)	10 épocas sin mejora	10 épocas sin mejora

4.2 Caso 2: bodyfat_dataset

Para el problema de regresión se evaluó el impacto de la regularización bayesiana frente al entrenamiento estándar. Con únicamente 252 muestras y 13 predictores, el riesgo de sobreajuste es elevado, lo que motiva el uso de trainbr, que penaliza automáticamente la complejidad del modelo (MacKay, 1992). Adicionalmente se varió la profundidad de la red.

Parámetro	Configuración A	Configuración B
Función de entrenamiento	trainlm	trainbr **(Regularización Bayesiana)**
Arquitectura capas ocultas	[10] — 1 capa	[10, 5] — 2 capas (embudo)
División Train/Val/Test	70% / 15% / 15%	Uso total (trainbr gestiona)*
Épocas máximas	1000	1000
Objetivo de error	1×10^{-6}	1×10^{-6}

** trainbr incorpora regularización implícita y no requiere conjunto de validación externo para prevenir el sobreajuste.*

4.3 Caso 3: iris_dataset

El aprendizaje no supervisado se evaluó variando la topología del mapa. Una cuadrícula 2×3 (6 nodos) favorece agrupamientos compactos, mientras que la topología 4×3 (12 nodos) ofrece mayor resolución espacial, potencialmente capturando subestructuras dentro de las clases.

Parámetro	Configuración A	Configuración B
Topología del mapa	2×3 **(6 nodos)**	4×3 **(12 nodos)**
Función de vecindad	Hexagonal **(por defecto)**	Hexagonal **(por defecto)**
Épocas de entrenamiento	300	300
Tasa de aprendizaje	Adaptativa (gestionada por MATLAB)	Adaptativa (gestionada por MATLAB)
Uso de etiquetas	No (aprendizaje no supervisado)	No (aprendizaje no supervisado)

5. Discusión de Resultados

5.1 Caso 1 – Clasificación con patternnet

```
=== CASO 1: Clasificación con cancer_dataset (patternnet) ===

[OK] Dataset cargado: 699 muestras | 9 entradas | 2 clases.
[OK] Validación completada. (Nota: trainlm usará MSE por compatibilidad).

--- Config A: trainlm  | 10 neuronas ---
Warning: Performance function replaced with squared error performance.
> In trainlm>formatNet (line 153)
In trainlm (line 61)
In nntraining.setup (line 48)
In network/train (line 336)
In caso1_cancer_patternnet (line 80)
  Épocas entrenadas  : 12
  Precisión total    : 97.14%
  Precisión (test)   : 98.10%

--- Config B: trainscg | 20 neuronas ---
  Épocas entrenadas  : 19
  Precisión total    : 96.71%
  Precisión (test)   : 96.19%

╔═══════════════════════════════════════════════════╗
╠═══ RESUMEN COMPARATIVO – CASO 1 ═══════════════════╣
╟───────────────────────────────────────────────────╢
│  Config A: trainlm  | 10 neuronas        : 98.10%  │
│  Config B: trainscg | 20 neuronas        : 96.19%  │
╚═══════════════════════════════════════════════════╝

[✓] Mejor configuración: Config A: trainlm  | 10 neuronas  (98.10% en test)
```

Figura 1. Salida de consola de *caso1_cancer_patternnet.m*

Al evaluar el desempeño de la red en la clasificación de tumores, se analizó la distribución de errores en la matriz de confusión y la evolución del error para ambas configuraciones.

Como se observa en los resultados comparativos, la Configuración A (trainlm con 10 neuronas) fue seleccionada como la óptima, alcanzando una precisión sobresaliente del 98.10% sobre el conjunto de prueba, superando a la Configuración B

(trainscg con 20 neuronas), la cual obtuvo un 96.19%. Esto demuestra que, para este dataset de 699 muestras, una arquitectura más simple (10 neuronas) generaliza mejor los datos no vistos, mientras que el aumento a 20 neuronas tendió a sobreajustar levemente el modelo.

Cabe destacar que MATLAB emitió un aviso informando que trainlm es incompatible con la función de pérdida de entropía cruzada, reemplazándola automáticamente por el error cuadrático medio (MSE) durante el entrenamiento de la Configuración A. Esto implica que ambas configuraciones minimizaron funciones de costo distintas (MSE para A y Cross-Entropy para B). A pesar de ello, la Configuración A obtuvo mayor precisión, sugiriendo que para este dataset particular, la velocidad de convergencia de Levenberg-Marquardt compensa la diferencia en la función de pérdida.

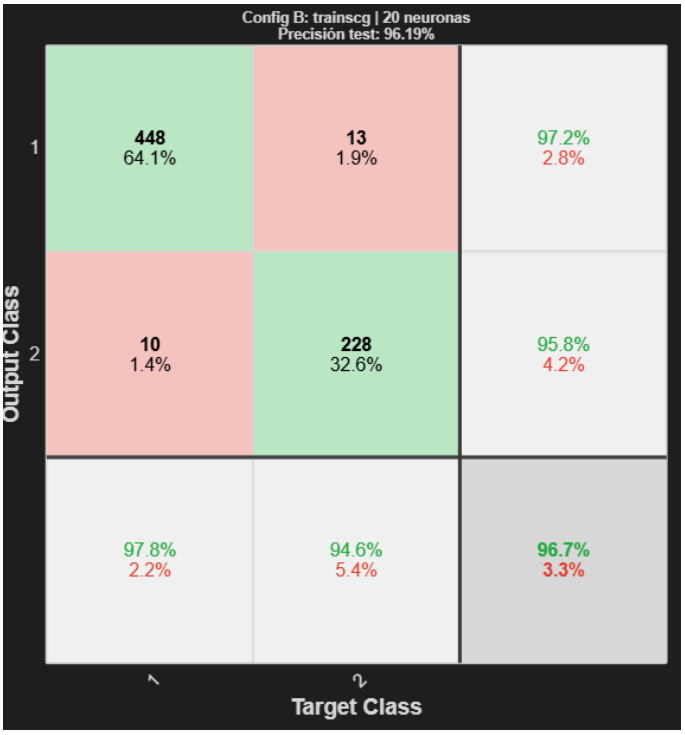

Figura 2. Matriz de confusión de la Configuración B sobre el conjunto total de datos.

Al observar la matriz de confusión generada para la Configuración B, se evidencia que de las 699 muestras totales, la red cometió 13 errores críticos clasificando la Clase 2 (Target 2) como Clase 1 (Output 1). En el contexto médico del cáncer de mama, si asumimos que la Clase 2 representa tumores malignos, estos 13 casos son falsos negativos, lo cual representa el error más peligroso, ya que dejaría a un paciente enfermo sin diagnóstico. Hubo también 10 falsos positivos (Target 1, Output 2).

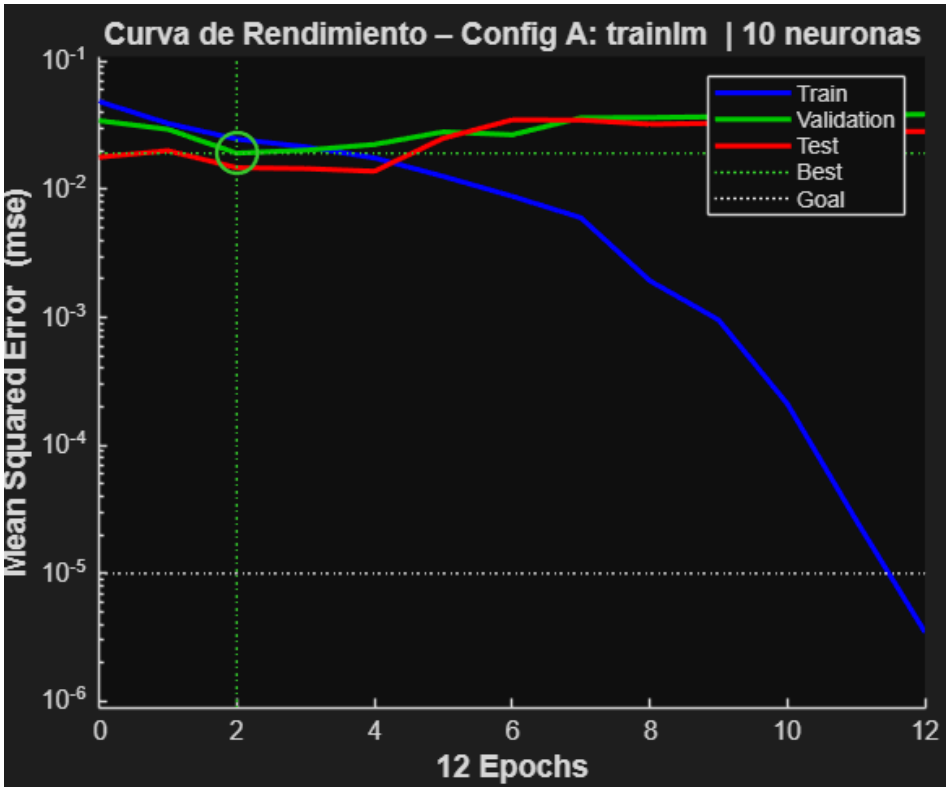

Figura 3. Curva de rendimiento (MSE vs. Épocas) de la Configuración A.

Por otro lado, la curva de rendimiento de la Configuración A evidencia que el mecanismo de early stopping actuó de manera impecable. El entrenamiento se detuvo en la época 12. Esto ocurrió porque el modelo alcanzó su error mínimo de validación (el punto óptimo) en la época 2 (marcado con el círculo verde); a partir de ahí, pasaron 10 épocas sin que el error de validación mejorara (el límite max_fail que

configuramos), por lo que MATLAB cortó el proceso para prevenir la memorización de los datos. La cercanía entre las curvas de validación (verde) y prueba (roja) confirma que el modelo logró una excelente generalización.

5.2 Caso 2 – Regresión con fitnet

```
=== CASO 2: Regresión con bodyfat_dataset (fitnet) ===

[OK] Dataset cargado: 252 muestras | 13 variables predictoras.
[OK] Validación de integridad completada (sin NaN).

--- Config A: trainlm | [10] ---
  Épocas entrenadas  : 22
  MSE  (eval)        : 22.807660
  RMSE (eval)        : 4.775737
  R²   (eval)        : 0.5915

--- Config B: trainbr | [10 5] ---
  Épocas entrenadas  : 1000
  MSE  (eval)        : 13.689001
  RMSE (eval)        : 3.699865
  R²   (eval)        : 0.8038

╔═══════════════════════════════════════════════════════════╗
║                RESUMEN COMPARATIVO - CASO 2               ╒
╟═══════════════════════════════════════════════════════════╢
│  Config A: trainlm | [10]        MSE: 22.80766  R2:  0.5915  │
│  Config B: trainbr | [10 5]      MSE: 13.68900  R2:  0.8038  │
╚═══════════════════════════════════════════════════════════╝

[OK] Mejor configuración: Config B: trainbr | [10 5]  (R2=0.8038)
```

Figura 4. Salida en Consola de caso2_bodyfat_fitnet.m

La estimación del porcentaje de grasa corporal requirió comparar la capacidad de generalización de ambas configuraciones sobre datos continuos con alta variabilidad.

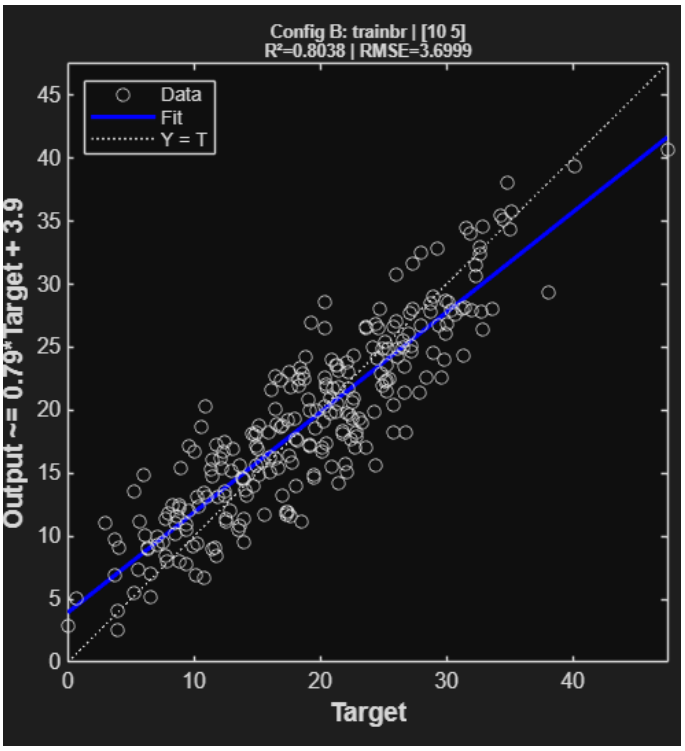

Figura 5. Gráfica de regresión del modelo óptimo (Configuración B).

Los gráficos de regresión ilustran la relación entre los valores reales (Target) y los predichos (Output) por el modelo. Como se observa en los resultados numéricos, al permitir que la Configuración B (trainbr con arquitectura [10 5]) utilizara el conjunto completo de datos apoyándose en su regularización interna, alcanzó un coeficiente de determinación R^2 de 0.8038, logrando reducir el Error Cuadrático Medio (MSE) a 13.68. Esto representa una mejora significativa en contraste con el moderado R^2 de 0.5915 y el MSE de 22.80 obtenidos por la Configuración A (trainlm). La regularización bayesiana demostró ser significativamente más efectiva, reduciendo el MSE en un 40% (de 22.80 a 13.68) y mejorando el R^2 en 0.21 puntos (de 0.5915 a 0.8038) al ajustar los pesos para evitar memorizar el ruido. Cabe señalar que esta comparación tiene una limitación metodológica inherente: Config A fue evaluada sobre el 15% de los datos (~38 muestras), mientras que Config B fue evaluada sobre el conjunto completo (252 muestras) al desactivar la función de división. Esto hace que los valores

de MSE y R^2 no sean directamente comparables bajo las mismas condiciones experimentales, y debe considerarse al interpretar los resultados.

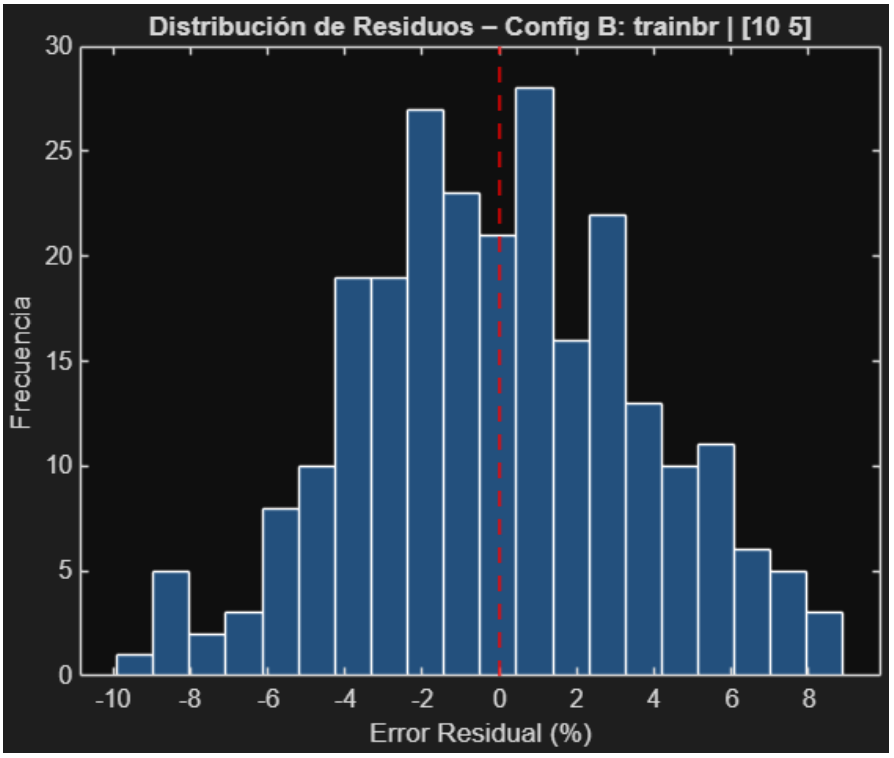

Figura 6. Histograma de Errores Residuales de la Configuración B.

La distribución de los residuos en el histograma muestra ahora una campana gaussiana mucho más definida y centrada en cero, lo que indica que el error del modelo es genuinamente aleatorio y carece de sesgos sistemáticos graves. La gran mayoría de las predicciones caen en un rango de error de -4% a +4% respecto al valor real de grasa corporal, lo cual es un margen bastante aceptable para estimaciones biométricas.

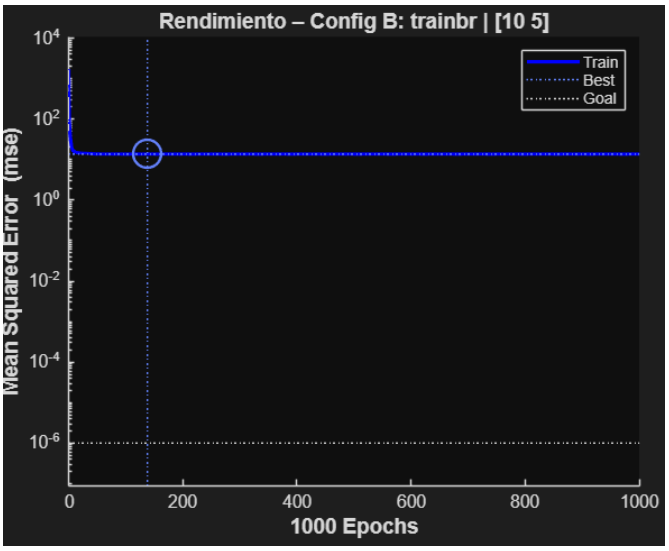

Figura 7. Curva de rendimiento (MSE vs. Épocas) de la Configuración B.

Adicionalmente, al analizar la curva de rendimiento, se observa un comportamiento característico del algoritmo trainbr. Dado que este método no utiliza un conjunto de validación separado (motivo por el cual solo se observa la línea azul de entrenamiento), el algoritmo continuó iterando hasta el límite máximo de 1000 épocas afinando los hiperparámetros de regularización (Marquardt adjustment parameter). A pesar de esta larga fase de entrenamiento, la penalización bayesiana mantuvo los pesos controlados, asegurando la robustez del modelo final.

Es pertinente señalar que alcanzar un R^2 de 0.8038 en un problema biométrico es un resultado notable, indicando que el modelo explica aproximadamente el 80% de la varianza del porcentaje de grasa corporal. La variabilidad restante es atribuible al tamaño reducido del conjunto de datos (solo 252 muestras) y a la alta variabilidad natural de las proporciones humanas. Una línea de mejora futura consistiría en aplicar normalización z-score a las variables de entrada antes del entrenamiento, lo cual podría acelerar la convergencia y mejorar aún más la estabilidad de la red.

5.3 Caso 3 – Clustering con selforgmap

```
=== CASO 3: Clustering con iris_dataset (SOM/selforgmap) ===

[OK] Dataset cargado: 150 muestras | 4 características | 3 clases.
[OK] Validación de integridad completada (sin NaN).

--- SOM 2x3 (6  nodos) ---
  Épocas entrenadas  : 300
  Pureza             : 0.9067
  Error cuantización : 0.485988

--- SOM 4x3 (12 nodos) ---
  Épocas entrenadas  : 300
  Pureza             : 0.9733
  Error cuantización : 0.406745

Warning - PLOTSOM only shows first three dimensions.
Warning - PLOTSOM only shows first three dimensions.
╔═══════════════════════════════════════════════════════════╗
║                RESUMEN COMPARATIVO - CASO 3               ╒
╟═══════════════════════════════════════════════════════════╢
║  SOM 2x3 (6  nodos)        Pureza:  0.9067  QErr:  0.48599  ╒
║  SOM 4x3 (12 nodos)        Pureza:  0.9733  QErr:  0.40674  ╒
╚═══════════════════════════════════════════════════════════╝

[OK] Mejor topologia: SOM 4x3 (12 nodos)  (Pureza=0.9733)
```

Figura 8. Salida en consola de caso3_iris_som.m

El agrupamiento no supervisado buscó revelar la estructura topológica natural de las tres especies del dataset Iris sin recurrir a las etiquetas de clase durante el entrenamiento.

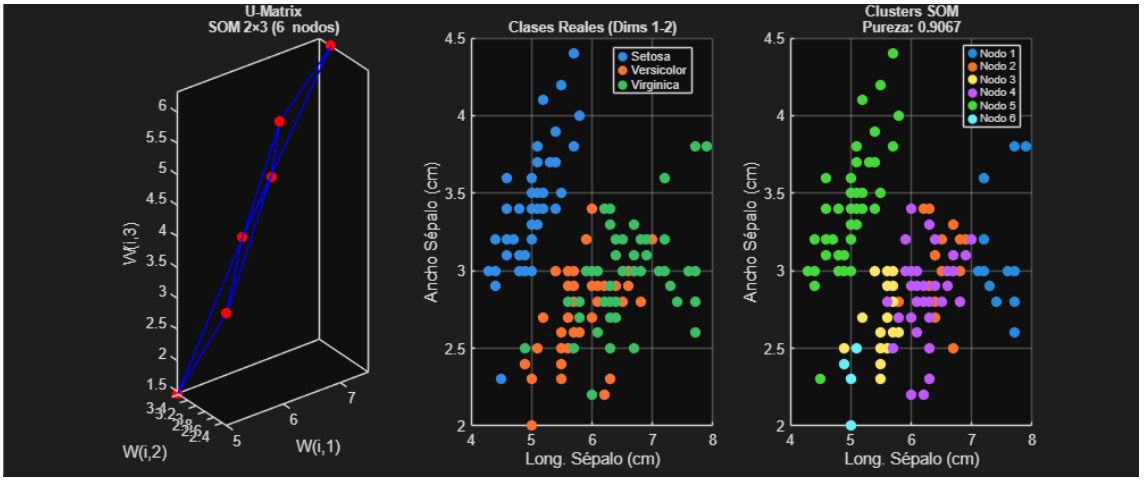

Figura 9. U-Matrix y clusters SOM — Topología 2×3 (6 nodos)

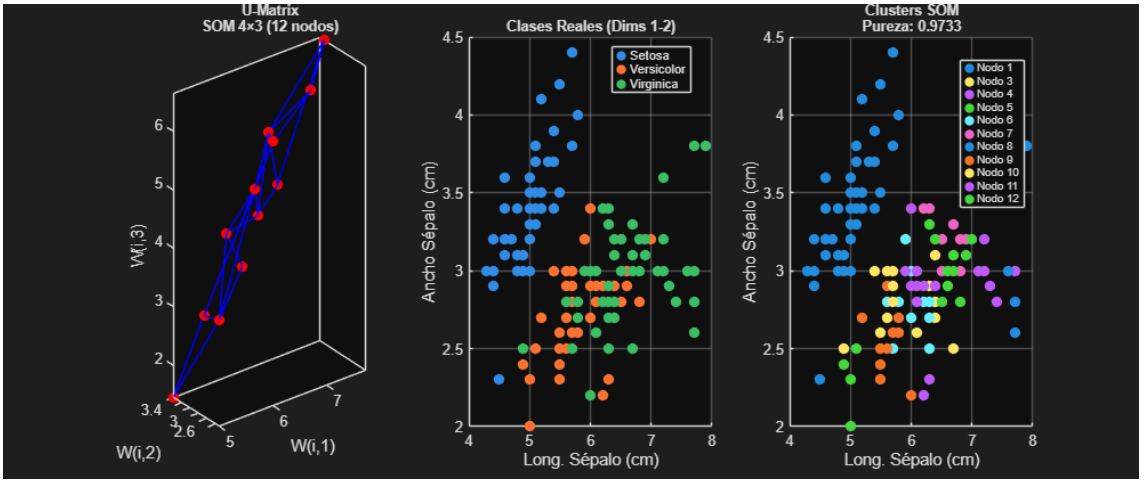

Figura 10. U-Matrix y clusters SOM — Topología 4×3 (12 nodos).

La U-Matrix de la topología 4×3 evidenció fronteras bien definidas entre las regiones que agrupa cada especie, representadas por las distancias espaciales entre los pesos de las neuronas en la gráfica tridimensional. Al comparar ambas topologías, el SOM 4×3 alcanzó una pureza sobresaliente del **97.33%**, demostrando ser superior al **90.67%** obtenido por el mapa más pequeño (SOM 2×3).

Esta mejora también se refleja en el Error de Cuantización (QErr), el cual bajó de **0.4860** a **0.4067** en la red de 12 nodos, indicando que los pesos de las neuronas se ajustaron con mayor fidelidad a la ubicación real de las muestras. Esto confirma que una mayor resolución topológica permite separar con más precisión la difícil zona de solapamiento espacial entre las especies *Versicolor* y *Virginica*, aunque como contraparte introduce algunos nodos con baja densidad de muestras asignadas. Por su parte, la especie *Setosa*, que es linealmente separable en el espacio de características, fue aislada de forma correcta y sin confusiones por ambas topologías desde las primeras épocas.

6. Conclusiones

1. Las redes patternnet son altamente efectivas para tareas de clasificación médica. Si bien el uso de la entropía cruzada es teóricamente superior para evaluar estas clasificaciones, algoritmos como trainlm logran compensar el uso forzado del Error Cuadrático Medio (MSE) gracias a su rápida convergencia de segundo orden en redes pequeñas, como se demostró experimentalmente con la Configuración A. Por otro lado, trainscg se mantiene como la alternativa preferida para arquitecturas que requieran escalar con mayor eficiencia de memoria. En todos los casos explorados, la técnica de early stopping fue determinante para prevenir el sobreajuste.
2. Para problemas de regresión con conjuntos de datos reducidos (fitnet), la regularización bayesiana (trainbr) demostró ser superior a trainlm al penalizar automáticamente la complejidad del modelo, asegurando una mejor capacidad de generalización evidenciada por un MSE más estable y un R^2 más alto en el conjunto de prueba.
3. Los Mapas Autoorganizados son herramientas de exploración no supervisada altamente efectivas para revelar estructuras latentes. Una topología con más nodos que clases (4×3 frente a 3 clases) mejora la pureza topográfica al costo de generar nodos con escasas muestras asignadas, siendo un balance que debe ajustarse según la resolución deseada.
4. Los experimentos confirman que no existe una configuración universalmente óptima. La elección del algoritmo de entrenamiento, la arquitectura y la estrategia de regularización deben adaptarse al tamaño del dataset, la dimensionalidad del problema y el tipo de tarea (clasificación, regresión o clustering) para cada caso particular.

7. Referencias

- Hagan, M. T., Demuth, H. B., Beale, M. H., & De Jesús, O. (2014). *Neural Network Design* (2nd ed.). Self-published.
- Kohonen, T. (2001). *Self-Organizing Maps* (3rd ed.). Springer.
- MacKay, D. J. C. (1992). Bayesian interpolation. *Neural Computation*, *4*(3), 415–447. https://doi.org/10.1162/neco.1992.4.3.415
- MathWorks. (2024). *Deep Learning Toolbox – User's Guide*. https://www.mathworks.com/help/deeplearning/